Worker Isolation: Responsibility and Resource Table

Branch `fix/worker-isolation`, base `35884ee`. The feature is gated by `WORKER_ISOLATION_ENABLED`, which defaults to **off**. With the gate off, the worker runs the unchanged in-process path from `35884ee`. With it on, each job attempt gets one `child_process.fork`. The **parent** is the BullMQ supervisor and the **child** runs the existing `runAnalysisPipeline`.

File:line references point at the branch head. Each "Test" entry names a test in `server/lib/job-isolation.test.ts` (29/29 pass, see `TEST_RESULTS.md`), or the opt-in integration suite `server/lib/job-isolation.integration.test.ts` (real Postgres + Redis + BullMQ + Chromium).

1. Full DB pool inventory (gap A)

Every `pg.Pool` that a worker replica can create:

Pool	Process	Max connections	Created	W
Main pool, parent	worker parent	**4** (`min(WORKER_ISOLATION_PARENT_PG_RESERVE=4, PG_POOL_MAX)`)	at import, lazy connect	`se` L2 ap ro pa on `Po`
Usage pool, parent	worker parent	**2** (`PG_USAGE_POOL_MAX`)	lazily, on first usage flush	`se` `us`
Main pool, child	each job child	**4** = `floor((PG_POOL_MAX 20 − 4) / WORKER_CONCURRENCY 4)`	lazy; 0 connections at startup (F1)	`jo` `ch` fo `PG` fo bu
	each job child	**0**: never created. Usage rows go to the parent over IPC	never	

Pool	Process	Max connections	Created	W
Usage pool, child				`ll` (`is` →
Redis / BullMQ	parent only	unchanged	parent only	ch `re` or

Totals per replica	Main	Usage	All PG
Baseline `35884ee` (gate off)	20	2	**22**
Isolation at `WORKER_CONCURRENCY=4` (deployed)	4 + 4×4 = 20	2 + 0 = 2	**22**
Isolation at `WORKER_CONCURRENCY=10`	4 + 10×1 = 14	2	**16** (C = floor(16/10) = 1)

The earlier version of this table said “20 = PG_POOL_MAX” and left out the usage pool. The correct total is **22 = 20 main + 2 usage**, the same as baseline. Test: **A1** checks the count from the real modules: `db.ts` `PG_POOL_MAX` evaluated in a worker-parent role, `childPgBudget()`, `llm-usage` `USAGE_POOL_MAX`, and `__usagePoolCreatedForTests()` = false in a child. **F1** checks `pool.totalCount = 0` in a booted child.

2. Deadlines (gap B)

The parent enforces both deadlines using its own timers. It does not wait for the child to cooperate.

Deadline	Source	Parent action	Class
Pipeline	`PIPELINE_TIMEOUT_MS`, same env as `pipeline.ts` L100 (code default 540 s; production 35 min)	timer from spawn (`job-isolation.ts` L201) → `onDeadline`: `revokeAttempt(lc,"pipeline_timeout")` + **immediate SIGKILL** of every group	`{erro` `out a` `failu` the sa proce Retri existi Comp failed `logPr` (`recor`
Job	`deadlineAt = claimTime + JOB_TIMEOUT_MS` (`worker.ts` L339;	timer at `lc.deadlineAt` (L202) → `revokeAttempt(lc,"job_watchdog_timeout")` + immediate SIGKILL	`Error` `timeo` `child`

Deadline	Source	Parent action	Class
	code default 600 s; production 40 min)		`deadli` non-r

Cleanup grace never extends past either deadline: - The cancel/revoke grace (`WORKER_ISOLATION_KILL_GRACE_MS` = 10 s, then SIGTERM, then +5 s SIGKILL) is pre-empted by `onDeadline`, which kills immediately. - The post-report exit grace (`WORKER_ISOLATION_EXIT_GRACE_MS` = 2 s) is also cut off by a deadline (L189-192). - `revokeAttempt` runs on the parent at the deadline, so the parent lifecycle refuses dispatch. Child score writes are refused by the existing SQL fence (`deadlineAt` + `attempt_token`).

Tests: - **B1**: child blocked forever with a 30 s grace configured. Resolved at 2529 ms (limit 2500 ms) with “Pipeline timed out”, revoked `pipeline_timeout`. Child, grandchild and detached descendant were all gone at settle. - **B2**: child keeps dispatching every 10 ms and ignores revoke and SIGTERM. It is SIGKILLed at `deadlineAt`. Its last raw (unfenced) fixture write landed within the timer + kill latency: +1 ms in the final run (−7 ms and −6 ms in earlier runs; test tolerance 50 ms). Nothing was written after that. Classified as non-retriable watchdog. Real score writes are additionally refused by the SQL `deadlineAt` fence. - **D6**: a cancel was in progress when the deadline arrived. The child was killed at the deadline, not after the 30 s grace, and the result stayed “Cancelled”.

3. Termination and parent death (gap C)

Event	Mechanism	Test
Graceful parent exit	`process.on("exit")` SIGKILLs all live groups (`job-isolation.ts` L141)	n/a
Parent SIGKILL / OOM (the exit hook does not run)	child-side **watchdog `worker_thread`** (`job-child.ts` L35) polls `process.ppid` every 250 ms on its own thread, so it also works while the child’s main loop is blocked. When the parent is gone it SIGKILLs registered descendant groups and the child’s own group. `disconnect` does the	**C** (blocked): supervisor SIGKILLed → child, grandchild and detached descendant gone in 41 ms. **C** (idle): 10 ms

Event	Mechanism	Test
	same when the child's loop is free (L41)	
Detached descendants (Chromium uses `detached:true`)	the child sends `pgroup` with Chromium's pid (`processor.ts` L1035 → `isolation-ipc.ts registerDescendantGroup`). The parent adds it to the child's kill set (`job-isolation.ts` L265)	B1, C, E2 (detached `sleep`); **integration**: real Chromium launched by two children via `fetchPdfViaBrowser`, peak 18 Chromium processes, 0 Chromium processes left after completion, 0 orphaned puppeteer profiles on the host

4. Retry table (gap D): same outcome as production

`classifyIsolatedOutcome` (`job-isolation.ts` L70) is the single place that decides how an isolated attempt ended. The decision then goes through `worker.ts`'s existing policy. That policy now lives in `server/lib/retry-policy.ts` (`NON_RETRIABLE_ERRORS`/`isRetriableError` moved verbatim) and `fenceCancelledResult` (the existing `worker.ts` Cancelled→watchdog fence, extracted with no behaviour change; called at `worker.ts` L465).

Event	Isolated outcome	`worker.ts` result	Matches in-process?	Test
Admin cancel (batch cancelled)	`"Cancelled"`	`failJob("Cancelled (batch cancelled)")`, **no retry**	yes	D row 1, test 2
Forced kill after cancel (child ignored it)	`"Cancelled"` (killReason stays `cancel`)	cancellation, no retry	yes	D row 2, **D6**
Ownership lost (revoke `ownership_lost`)	`"Cancelled"`, batch not cancelled → fenced to `"Job watchdog timeout …`	non-retriable. `failJob` with `transitioned:false` is a stale no-op (`worker.ts`	yes	D row 3, test 3

Event	Isolated outcome	`worker.ts` result	Matches in-process?	Test
	`(attempt revoked: ownership_lost)"`	L581-584, L689), so there is **no stale transition and no duplicate retry**		
Job watchdog deadline	`Error("Job watchdog timeout: …")`	**final** (non-retriable)	yes	D row 4, **B2**
Pipeline deadline	`{error:"Pipeline timed out after Ns"}`	**retriable** (existing policy, kept and verified)	yes, identical string	D row 5, **B1**
Crash (non-zero exit, no kill requested)	`Error("Isolated job child exited unexpectedly (code=1 …)")`	retriable up to `MAX_RETRY_ATTEMPTS`, non-NULL `last_error`	same handling as an in-process thrown error	D row 6, test 4
OOM / external SIGKILL	same as crash	retriable	same	D row 7
Child reported before any kill	exactly what it reported	unchanged	yes	D row 8, **M1**
Child reported **after** a terminal decision (deadline / cancel / revoke)	the terminal decision; the report is discarded (logged, `lateReportsDiscarded++`)	as the decision row above	yes (in-process, the watchdog race also wins once it has fired)	**M2, M2b, M3**
Child exited but its process groups are not confirmed gone	`Error("Isolated job child cleanup unconfirmed: …")` even if it reported success	**non-retriable** (`retry-policy.ts` L31); job fails, permits stay withheld	n/a (new failure mode that only exists with a child)	**Q1**

5. Permits and background work (gap E)

Rule	Mechanism	Test
A child cannot report success while its background work is alive	`result/error` only records the outcome (`onReported`, L223). Settlement happens on child **exit**. Exit grace is 2 s, then SIGKILL. On exit: kill groups → poll until gone (≤ 2 s) → `releaseAll` → settle (L289-304)	**E2/E3**: delivered at 3517 ms. At delivery the child, grandchild, held proc permit and live set were all gone/0
Cancelled queued IPC acquisitions are removed	`cancelAcquire` aborts the parent's LLM waiter (L255-259). A grant arriving after withdrawal/exit is handed straight back (L249). On exit, ungranted waits are aborted (L208)	**E1**: pending 0 after abort while 8 were held. **E1b**: child dies with 8 held + 1 queued → all returned
Permits are not released before the resource has stopped	`releaseAll` runs only after the child has exited **and** the zombie-aware probe confirms every registered group is gone. If not confirmed in `WORKER_ISOLATION_REAP_WINDOW_MS` (2 s), the permits are **withheld** in quarantine (section 8)	**E2/E3** `releasedWhileAlive=false`; **Q1/Q2**
Subprocess limit = baseline 6 per replica	`MAX_CONCURRENT_SUBPROC` default 6 (`processor.ts` L557, unchanged from `35884ee`), brokered by the parent	**E4**: 3 children × 4 → max 6
LLM 8 / browser 1 per replica	parent `llmSlotBroker` / `processorSlotBroker`	test 5: max 8 / 1

6. Child startup (gap F)

The child entry is `job-child.ts`. It never imports `worker-main.ts` (which is where `initializeDatabase`, `cleanupOnStartup` and `startReconciler` run, L79-98).

Test **F1** recorded the child's state at boot: - Handles at entry: `PipeWrap, Timeout`. - After importing `pipeline.ts`: `PipeWrap, Timeout`. The only handle is the IPC pipe (fd3), and there are no sockets. - `pg` totalCount 0, and no usage pool. - Server modules loaded: only `server/storage.ts` and `server/db.ts`. `db.ts` builds a lazy `Pool` object

with no connection. - Not loaded: `redis.ts`, queue, `startup-cleanup`, reconciler, `worker*.ts`, `reconcile-resurrect`, `ioredis`, `bullmq`. `redis.ts` throws at import when `REDIS_URL` is unset, and the test unsets it.

7. What still runs on the parent (gap G)

“The parent never runs pipeline CPU” is true. It is narrower than “the parent cannot be blocked”. Work still on the parent loop:

Parent work	**Cost / bound**	**Can it block?**
BullMQ consume + lock renewal, Redis cancel cache	network I/O	no CPU risk
claim, heartbeat (×4), ownership watch (×4, 30 s), cancel poll (500 ms per child)	short single SQL / timer	no
`fork()` per attempt	G1: max 2.6-9.1 ms across runs (7.6 ms in the final run)	synchronous, bounded
`structuredClone` of init opts (company/ framework/ measures/ BatchFetchState)	proportional to payload	yes, if payloads grow large
IPC handlers (acquire/release/ usage/stage/ pgroup)	G1: max ≤ 0.10 ms per message	no
permit broker, usage-buffer flush (usage pool 2)	O(1) / batched INSERT	no
BatchFetchState merge at job end	O(hosts)	small
terminal writes, retry, batch counters (unchanged `worker.ts`)	SQL	no
GC of the parent heap	small heap (no pipeline objects)	short pauses
host CPU saturation (4 children on 2 vCPU)	children run at **nice 10** (L162)	mitigated, not eliminated

Test **G1**: 4 CPU-bound children on this 2-CPU host gave parent max lag **10.4 ms**, forks 4, max fork 7.6 ms, 12 messages, max handler 0.10 ms (final run; earlier runs 4.5 / 4.6 ms lag).

Preserved unchanged: scoring rules, BM25, research and scoring code, `PG_POOL_MAX`, `PG_USAGE_POOL_MAX`, `JOB_TIMEOUT_MS`, `PIPELINE_TIMEOUT_MS`, `WORKER_CONCURRENCY`, `LLM_MAX_CONCURRENCY`, `MAX_CONCURRENT_BROWSER`, `MAX_CONCURRENT_SUBPROC`. The gate stays off by default.

8. Unconfirmed descendant cleanup: quarantine (FIX 1)

Step	Mechanism (`job-isolation.ts`)	Test
Liveness probe	`probeGroups` (L115): scans `/proc/*/stat`, reads pgrp (field 5) and state (field 3). A group counts as live only if it has a member whose state is not `Z`. `kill(-g,0)` alone cannot tell a zombie from a live process. If `/proc` is unreadable, it falls back to `kill(-g,0)` with `reliable:false` and treats every visible group as live (conservative: capacity stays withheld)	**Q3**: group holding only an unreaped zombie → terminated, clean success, no quarantine, no 2 s wait
Exit handler	after the child exits, SIGKILL groups, then poll the probe for up to 2 s (L427)	E2/E3
Not confirmed	quarantine entry {child pid, groups, reason, withheld browser/proc/llm grants}; the grants are **not** returned; queued (ungranted) waits are aborted; the attempt settles as `Isolated job child cleanup unconfirmed …` (L436-442), which is non-retriable and is **never** a clean success, even if the child reported success	**Q1**: reported success → rejected, browser permit not reassigned while quarantined
Background monitor	`ensureQuarantineMonitor` (L158) re-probes every 5 s (`WORKER_ISOLATION_QUARANTINE_POLL_MS`) and releases the withheld permits only once termination is confirmed (`releaseQuarantined`, L150)	**Q1** (released after confirmation)
Still alive	quarantine persists; capacity stays reduced	**Q2**
	`recoverQuarantinedChildren(pid?)` (L182): SIGKILLs the groups once	**Q2**

Step	Mechanism (`job-isolation.ts`)	Test
Deliberate recovery	more and returns the permits regardless (operator action; nothing calls it automatically)	
Parent exit	the exit hook SIGKILLs quarantined groups as well as live ones	n/a

9. Monotonic terminal decision (FIX 2)

- `killReason` is set **once**, by the first terminal event: pipeline deadline, job deadline, cancel, or revoke (`onDeadline` L283). A later deadline only brings the kill forward; it does not change the reason.
- `onReported` (L337) accepts a child's result/error only if `killReason` is still null. Otherwise the report is logged (`[WORKER-ISOLATION] … reported AFTER terminal decision … discarded`), counted (`lateReportsDiscarded`), and kept in a 50-entry diagnostic ring. It never changes the outcome.
- A report accepted first keeps precedence, even if a deadline fires before the child exits (the deadline still kills, so nothing outlives it).
- Tests use ordering barriers, not sleeps: the fixture waits for a `FIXTURE_GO` file, and the test hook `setSuppressDeadlineKill` lets the deadline make its decision (and revoke) without SIGKILL so that the child can report afterwards. **M1** report→deadline = success; **M2** job deadline→report = job watchdog timeout (non-retriable); **M2b** pipeline deadline→report = pipeline timeout (retriable, existing policy); **M3** cancel→report = Cancelled.

10. Browser permits vs Chromium instances

Item	Baseline `35884ee` (in-process)	Isolation on
Concurrent browser page fetches per replica	`MAX_CONCURRENT_BROWSER` = 1	1, brokered by the parent (test 5; integration broker state `browser.active=0` after completion)
Chromium **instances** per replica	1 shared instance for the whole worker	**1 per job child that used the browser**, alive until that child exits (the shared-

Item	Baseline `35884ee` (in-process)	Isolation on
		browser singleton is per process). Up to `WORKER_CONCURRENCY` instances (4 in production)
Measured (integration, 2 children, real Chromium)	n/a	peak **18 Chromium processes**, summed RSS **2401 MB** (sum of per-process RSS; shared pages are counted more than once, so this overstates real memory); all descendants together 2801 MB; parent 210-238 MB

The permit limits concurrent fetches, not instances. The memory cost of one Chromium per child is the main staging risk and is **not** solved by this branch.